

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE CODE: CSA1402

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 20

Total Marks:100

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I V.VANDHANA VARSHA(192524265)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:V.VANDHANA VARSHA(192524265)

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Explanation	25%				
Application	20%				
Organization	15%				
Accuracy	10%				
Clarity	5%				

Faculty Incharge

SHORT ANSWER TEST

1. Construct language for the following Regular Expression:2

$$(a|b)^*a(a|b)|(a|b)$$

2. a) Illustrate the predictive parser for the following grammar. 4

$$S \rightarrow (L) | a$$

$$L \rightarrow L, S | S$$

3. Analyse, is it possible, by modifying the grammar in any way to construct predictive parser for the language of

$$S \rightarrow SS^+ | SS^* | a$$

String: "aa+a" 4

4. Eliminate left recursion from the grammar $S \rightarrow Sa|Sb|c$ 2
 5. The following grammar contains common prefixes, making it unsuitable for predictive parsing.

$$A \rightarrow abcD | abcE | f$$

Apply left factoring to remove the common prefixes and rewrite the grammar.5

6. Consider the following statement:10

$$Degree - F = Degree - C * 1.8 + 32$$

Show the representation of this statement after each phase of the compiler and explain.

7. Consider the following C code fragment:4

```
sum = 0;
```

```
for (i = 1; i < 5; i++)
    sum = sum + i;
```

Show the representation of the above code after each phase of the compiler.2

8. Consider the following grammar.

$$S \rightarrow aABb, A \rightarrow c|\epsilon, B \rightarrow d|\epsilon$$

Calculate FIRST and FOLLOW, Obtain the Predictive Parsing Table.

not. Show the parser moves for the input string **acdb**.

9. Write a simple Lex program to recognize and count identifiers in a given input.2
 10. Write the rule to Eliminate Left Recursion and Left Factoring.2
 11. What are the actions performed by a shift-reduce parser? Explain each action with a suitable example.3
 12. Consider the following grammar:

$$S \rightarrow pBB \rightarrow qB|r$$

Find the leftmost derivation and construct the parse tree for the input string pqqrr.3

13. Consider Grammar: $E \rightarrow E+T | T$ $T \rightarrow T * F | F$ $F \rightarrow (E) | id$
 Eliminate left recursion and find FIRST and FOLLOW 5

14. **For the following grammar, determine whether the string abcc is accepted using shift-reduce parsing. Show all shift and reduce operations.**

$$S \rightarrow ABA \rightarrow aA|aB \rightarrow cB|c$$

15. For the following grammar, verify whether the string **abbc** is valid using shift-reduce parsing.10

$$S \rightarrow aABA \rightarrow bA|bB \rightarrow c$$

Show the complete sequence of stack, input, and parser actions.

16. Consider the following grammar:10

$$S \rightarrow SS \rightarrow aABA \rightarrow bA|cB \rightarrow d$$

- Construct the canonical collection of LR(0) items.
- Construct the SLR(1) parsing table.

17. Consider the following grammar.

$$S \rightarrow iEtSS|a, S \rightarrow eS|\epsilon, E \rightarrow b$$

Obtain the Predictive Parsing Table. 5

18. Do left factoring in the following grammar: 3

$$A \rightarrow aAB|aA|a$$

$$B \rightarrow bB|b$$

19. Construct the LL(1) Parsing Table for the following grammars.7

$$S \rightarrow ABCA \rightarrow a|\epsilon B \rightarrow r|\epsilon C \rightarrow b|\epsilon$$

20. Construct the LL(1) Parsing Table for the following grammars.7

$$S \rightarrow 1S\#|qABCA \rightarrow a|bbDB \rightarrow a|\epsilon C \rightarrow b|\epsilon D \rightarrow c|\epsilon$$

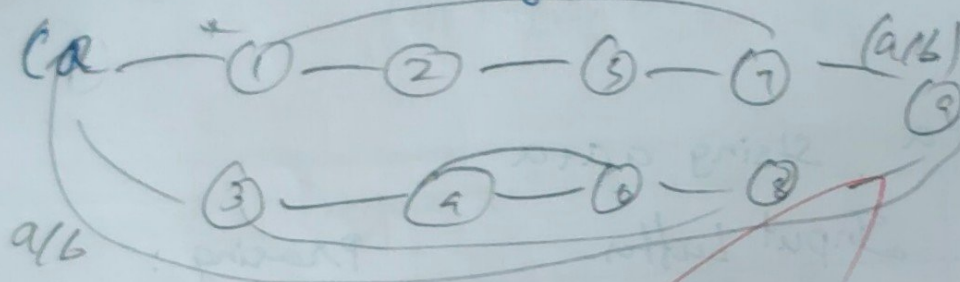
RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers

Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

CO2-AT-2 Short answer Test

1) $(a/b)^*a(a/b)(a/b)$ - Regular expression



60
100

2) predictive parser

$S \rightarrow (L) | a$

$L \rightarrow L, S | \epsilon$

$S' \rightarrow a$

$S \rightarrow (L)$

First

$F(S) = \{(, a\}$ follow = $\{\$, \epsilon\} = S$

$L' \rightarrow S$ $F(S') = \{a$ follow = $\{\$, \epsilon\} = (L$

$L' \rightarrow \epsilon$ $F(L') = \{\epsilon\}$

$L \rightarrow SL'$ $F(L) = \{(, a\}$ follow = $\{\$, \epsilon\} = (L$

3) predictive parser

$S \rightarrow SS+ | SS^* | a$

string "a+a"

$S \rightarrow a$ first $S = \{a\}$

$S \rightarrow S^* S$ first $L = \{(, a\}$

$S \rightarrow S+ S$ first

$S \rightarrow SS^+ a$

NON	ϵ	ϵ	ϵ	ϵ
S	$S \rightarrow a$	$S \rightarrow a$		
L	$L \rightarrow S$		$L \rightarrow S$	$L \rightarrow S$
L	$L \rightarrow SL'$		$L \rightarrow SL'$	$L \rightarrow S'$

4) eliminate left recursion $S \rightarrow Sa | Sb | c$

$S \rightarrow Salsb|c$

$S \rightarrow c$

$S \rightarrow a * S$

$S \rightarrow b * S$

3) $S \rightarrow ss + | ss * | a$ string $aa + a$

Stack	Input buffer	Pracing :
\$	aa+a\$	Shift
\$a	a+a\$	Shift
\$aa	+a\$	Shift
\$ss	+a\$	reduce $S \rightarrow a$
\$ss+	a\$	Shift
\$s	a\$	reduce $S \rightarrow ss +$
\$sa	\$	shift
\$ss	\$	reduce $S \rightarrow s \rightarrow a$
\$s	\$	accepted

Predictive parsing

$A \rightarrow abcD / abcEF$

$A \rightarrow abcD / F'$ First

Follow

$A' \rightarrow abcE / F'$ $f(A) = \{ (a, E') \}$

$f(A) = \{ \$, a \}$

$f(A') = \{ \epsilon, b \}$

$f(b) = \{ \$, b \}$

2

b) Degree-F = Degree-C * 1.8 + 32

explain in the phase of compiler

- 1) lexical analysis
- 2) semantic analysis
- 3) syntax analysis
- 4) Code generator
- 5) Code optimisation
- 6) Code generator

syntax analysis

$id' = id + id2$

$id = id2$
 $\rightarrow temp = 1.8 + 32$
 $temp2 = Degree - C$
 $temp3 = temp2 * temp$

$\langle Degree - F \rangle = \langle Degree - C \rangle \langle * \rangle \langle 1.8 \rangle \langle + \rangle \langle 32 \rangle$

$Degree - F = id$ identifies

semantic analysis

$id' = id * 1.8 + 32$

$Degree - C = id$ identifies

1.8

constant

32

constant

* mul

multiplicat factor

+

Addition

7) Sum = 0;
 for (i=1; i<5; i++)
 sum = sum + i

Phase of compiler

Lexical analysis:

It analyse about the invalid character
 for ex: i = 1a, sum = id, =, sum + i
 = sum = id₂ + sum + i

Syntax analysis

It analyse about the syntax error

for (i=1; i<5; i++) missing colon

Semantic analysis

It analyse about the word instead of integer

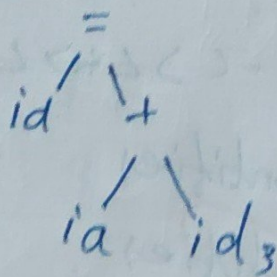
for (i = one ; i < Five ; i++)

~~S → aABb~~

~~A → c | ε~~

~~B → d | ε~~

Syntax analysis



Semantic analysis

id =, id₂ + id₃

Intermediate code generator

temp = id₃

id = temp + id₂

id = temp₂

temp = temp₂ + temp₁

code optimizing

temp = id₃

temp = id₂

temp₃ = temp₂ + temp₃

code

MOV r, id₃, r₂

MOV r, id₂, r₁

MOV r, id₁, r₃

to find First-Follow
 3) $S \rightarrow aABL$,
 $A \rightarrow c/\epsilon$ $A \rightarrow c$ $B \rightarrow d/\epsilon$
 $B \rightarrow d/\epsilon$ $A \rightarrow \epsilon$ $B \rightarrow \epsilon$

First

$$\text{First}(S) = \{a\}$$

$$\text{First}(A) = \{c, \epsilon\}$$

$$\text{First}(B) = \{d, \epsilon\}$$

Follow

$$\text{Follow}(S) = \{\$, a\}$$

$$\text{Follow}(A) = \{\$, c, a\}$$

$$\text{Follow}(B) = \{\$, d, c, a\}$$

Non Terminal	\$	a	c	d	ϵ
S	$S \rightarrow aABL$ $S \rightarrow aABL$				
A	$A \rightarrow c$	$A \rightarrow c$	$A \rightarrow c$		$A \rightarrow \epsilon$
B	$B \rightarrow d$	$B \rightarrow d$	$B \rightarrow d$	$B \rightarrow d$	$B \rightarrow \epsilon$

- 9) Lex program to recognize and count identifies to give input
- 10) rule to eliminate Left recursion and Left factoring
- 11) Shift reduce parser.

9) Lex program to count identifies.

1a)

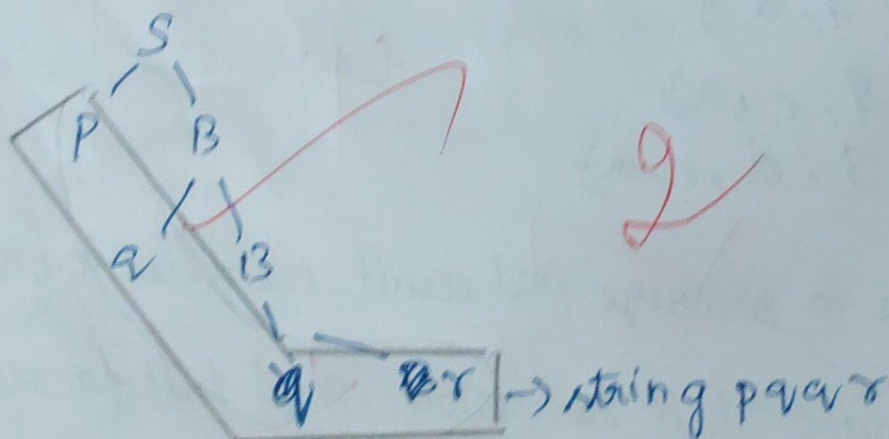
Consider the following

$S \rightarrow pB$

$B \rightarrow aB \mid \epsilon$

string $paaaa$

Parse tree



b)

$S \rightarrow AB$

$B \rightarrow aA \mid a$

$B \rightarrow cB \mid c$

string $abcc$ is ~~accepted~~ using shift
reducing parsing

$S \rightarrow aAB$

$A \rightarrow bA \mid b$

string $abcc$ is valid using shift. reducing
parsing

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

Non Terminal	\$	+	*	(	)	ϵ
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'	$E' \rightarrow +TE'$ $E' \rightarrow \epsilon$			$E' \rightarrow +TE'$		
T	$T \rightarrow FT'$	$T \rightarrow FT'$		$T \rightarrow FT'$		
T'	$T' \rightarrow *FT'$ $T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow *FT'$		
F	$F \rightarrow id$	$F \rightarrow id$	$F \rightarrow id$	$F \rightarrow id$		

$First(E) = \{ (, id \}$

$First(E') = \{ +, \epsilon \}$

$First(T) = \{ (, id \}$

$First(T') = \{ *, \epsilon \}$

$First(F) = \{ (, id \}$

$Follow(E) = \{ \$,) \}$

$Follow(E') = \{ \$,) \}$

$Follow(T) = \{ +, \$,) \}$

$Follow(T') = \{ +, \$,) \}$

$Follow(F) = \{ *, +, \$,) \}$

$S \rightarrow AB \quad B \rightarrow aA \mid a \quad B \rightarrow cB \mid c \quad \text{input} \rightarrow abcc$

Stack	Input Buffer	passing.
\$S	abcc\$	reduce $S \rightarrow AB$
\$AB	abcc\$	shift $A \rightarrow aA$ (shift
\$aAB	bcc\$	reduce $aA \rightarrow A$
\$AB	bcc\$	Shift
\$ABd	cc\$	Shift.
\$ACB	cc\$	Shift
\$ACBd	cc\$	reduce.
\$ABd	c\$	reduce.
\$AB\$AB	\$	Shift not accepted.

5)

$S \Rightarrow abc$ $A \rightarrow bA/b$

$B \rightarrow c$

abbc - Input string

Stack	Input buffer	passing
\$s	abbc\$	reduce s-abc
\$abc	abbc\$	Shift a
\$aabc	bbc\$	Reduce bA $\rightarrow$ A
\$aAbc	bbc\$	Shift
\$as	bbc\$	Reduce abc $\rightarrow$ S
\$asb	bc\$	Shift
\$asbb	c\$	Shift
\$asbA	c\$	Reduce A $\rightarrow$ BA
\$aSAc	\$	Shift
\$ASAC	\$	not accepted.

10) Reduce-Shift parsing:

Shift reduce parsing It is used in the stack implementation.

There are shift, reduce, accepted.

The shift are used when the variable are greater than or less than the next variable.

~~<id> * <+>~~

<id> <* > <+ > <(> < > <non term>

shift example:

\$ a+a\$

\$a +a\$ shift

- Reduce occurs when the values can be change to the condition given in the problem grammar

For example

$(S \rightarrow a)$

\$a +a\$ shift

\$S +a\$ reduce $S \rightarrow a$.

- Accepted when the stack and input buffer gets the same symbol

For example

\$ \$ accepted.

20) Construct LL(1)

$S \rightarrow 1S \mid \epsilon$ / $VABC$

$A \rightarrow a \mid b \mid D$

$B \rightarrow a \mid \epsilon$

$C \rightarrow b \mid \epsilon$

$D \rightarrow C \mid \epsilon$

(17)

$S \rightarrow (E \mid S'S' \mid a, S' \rightarrow \epsilon \mid \epsilon,$

$E \rightarrow b \rightarrow$ predictive parsing

18) Do left factoring

$A \rightarrow$

16. $S' \rightarrow S$
 $S \rightarrow aAB$
 $A \rightarrow bA | \epsilon$
 $B \rightarrow d$
 $\text{Goto}(I_0, S') = I_1$

$S' \rightarrow S$
 $S \rightarrow aAB$
 $\text{Goto}(I_0, A) = I_2$
 $A = \cdot bA | \epsilon$

$A \cdot$
 $\text{Goto}(I_0, B) = I_3$
 $B = \cdot d$
 $B \rightarrow d$

$\text{Goto}(I_0, \epsilon) = I_4$
 $A \rightarrow \epsilon$

$\text{Goto}(I_1, a) = I_5$

$S \rightarrow aAB$

$S \rightarrow a \cdot cd$

$S \rightarrow \cdot acd$

$\text{Goto}(I_2, b) = I_6$

$A \rightarrow bA$

$A \rightarrow \cdot bA$

$\text{Goto}(I_3, d) = I_7$

$B \rightarrow d$

$\text{Goto}(I_4, \epsilon) = I_8$

$A \rightarrow \epsilon$

$\text{Goto}(I_4, \epsilon) = I_8$

$A \rightarrow \epsilon$

$\text{First}(S') = \{a\}$

$(S) = \{a\}$

$(A) = \{b, \epsilon\}$

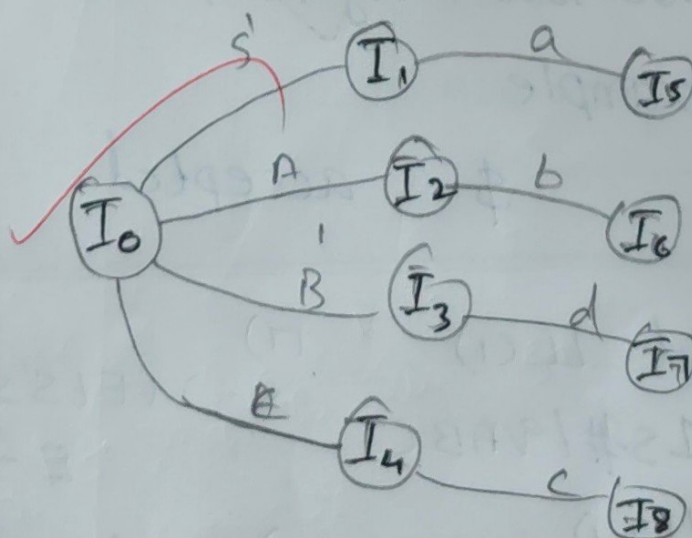
$(B) = \{d\}$

$\text{Follow}(S') = \{\epsilon\}$

$(S) = \{\$, a\}$

$(A) = \{\$, b, \epsilon, d\}$

$B = \{\$, b, \epsilon, a, d\}$



	\$	a	b	c	d	e	s	A	B
I_0									
S_1				S_4			1	2	3
I_2		S_5							
I_3			S_6						
I_4					S_7				
I_5				S_8					
I_6	r_1	r_1							
I_7	r_2	r_2		r_2	r_2				
I_8	r_3			r_3					